Overview of Chipyard

Dennis Frank

Installation of Chipyard

WORK IN PROGRESS

The installation of Chipyard is described in `https://chipyard.readthedocs.io/en/latest/Chipyard-Basics/Initial-Repo-Setup.html`. Some additional steps are added by ChatGPT. These steps were performed on a system with Ubuntu 24.04.3 LTS, a differnt configuration could require additional steps!

1. Step: Dependencies:

Scala:

```
curl -O -L https://github.com/chipsalliance/chisel/releases/latest/download/
chisel-example.scala

curl -sSLf https://scala-cli.virtuslab.org/get | sh
```

Termurin:

```
sudo apt install -y wget gpg apt-transport-https

echo "deb https://packages.adoptium.net/artifactory/deb
$(awk -F= '/^VERSION_CODENAME/{print$2}' /etc/os-release) main" |
sudo tee /etc/apt/sources.list.d/adoptium.list

wget -qO - https://packages.adoptium.net/artifactory/api/gpg/key/public |
gpg --dearmor | sudo tee /etc/apt/trusted.gpg.d/adoptium.gpg > /dev/null

apt update

apt install temurin-17-jdk
```

Ubuntu Packages:

```
sudo apt-get install autoconf automake autotools-dev curl libmpc-dev libmpfr-dev libgmp-dev libusb-1.0-
sudo apt install libguestfs-tools
```

The following steps are neccesary:

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh

bash Miniconda3-latest-Linux-x86_64.sh

source ~/miniconda3/etc/profile.d/conda.sh
```

After this clone the repository from chipyard and execute the installation, caution a lot of space is needed! Follow the steps:

```
    git clone https://github.com/ucb-bar/chipyard.git

cd chipyard

./scripts/init-submodules-no-riscv-tools.sh

source  ./env.sh

./build-setup.sh riscv-tools
```

Simulation

RTL Simulation

The RTL simulation is done with **Verilator**. With this simulation the actual hardware architecture of Rocket Chip is simulated. Every instruction is executed just like the hardware working in cycles. That's why the behavior is cycle-accurate. The main disadvantage is, that the simulation takes a lot of time, especially for more komplex code or bigger software.

First of all you will generate an executable which is used to simulate the chip, in this case the Rocket Chip (but there are other configurations available).
The following command generates the executable, use this command in the chipyard directory. After this the executable can be found within chipyard/sims/verilator:

```
make -C sims/verilator verilog CONFIG=RocketConfig
```

After this it is possible to simulate own code written in C/C++. Therefore you need to compile your code to code that is useable for the simulation. In general you can use the following command, it will create another file with ".elf" as ending:

```
riscv64-unknown-elf-gcc -o <name>.elf <name>.c
```

Now you can use the generated file within the simulation. For this enter sims/verilator and execute the following command:

```
./simulator-chipyard.harness-RocketConfig pk
    "the-whole-path-to-your-.elf-file-starting-at-home-directory"
```

The last command includes "pk" which stands for "Proxy Kernel". It is a tiny OS provided by the RISCV group and handles basic tasks like providing an evironment to deal with system calls. With these steps it is possible for example to write a simple "Hello World" program an run it with the verilator simulation. The result could look like this:

```
(/home/dennis/Dokumente/Projekte/chip/chipyard/.conda-env) dennis@hawk:~/Dokumen
te/Projekte/chip/chipyard/sims/verilator$ ./simulator-chipyard.harness-RocketCon
fig pk ~/Dokumente/Projekte/test-binaries/hello-riscv
[UART] UART0 is here (stdin/stdout).
Hello world!
- /home/dennis/Dokumente/Projekte/chip/chipyard/sims/verilator/generated-src/chi
pyard.harness.TestHarness.RocketConfig/gen-collateral/TestDriver.v:158: Verilog
$finish
```

There are already different examples of code within chipyard, that can be run by verilator. More about this can be found under (2.1.5):
https://chipyard.readthedocs.io/en/stable/Simulation/Software-RTL-Simulation.html
It is possible to run the example code with the last command mentioned above. There is actually no use of "pk", you can leave this out in this case.

NOTE: It could be neccesary to execute *cmake .* after executing *make* in the "tests" directory of chipyard. After this you can go on with the instructions in the documentation.

Functional Simulation

This simulation can be accomplished with **Spike** (or with Qemu). In this case only the RISCV instruction-architecture is simulated. This means that the instructions will be executed following the RISCV specification. It shows if the simulated program works correctly in terms of logic and architecture.
The command to use spike is:
spike pk "the-whole-path-to-your-.elf-file-starting-at-home-directory"

You don't need to add the whole path if you are in the directory which contains the file you want to execute. In this case you just need to add the file. If you want to execute the example code in tests from the chipyard directory with spike, leave out the "pk".

Algorithms in Simulation with Spike

Fast Fourier Transformation

The Fast Fourier Transformation was implemented with the help of FFTW (`https://www.fftw.org/`). In my case, it was neccesary to install it manually with the following commands:

```
wget http://www.fftw.org/fftw-3.3.10.tar.gz

tar xzf fftw-3.3.10.tar.gz

cd fftw-3.3.10

export CC=riscv64-unknown-elf-gcc

export AR=riscv64-unknown-elf-ar

export RANLIB=riscv64-unknown-elf-ranlib

export HOST=riscv64-unknown-elf

./configure --host=$HOST --enable-static --disable-shared --prefix=$(pwd)/install

make -j4

make install
```

To follow the instructions you need the following files: image.py, fft_output.txt, image_data.h, fftw.c, gen_freq_pic.c and an image (e.g. test.png). All of these files should be in the same directory.
The first step is to use the Python script to transform a picture into an array of values with the following command. You will be asked for a name of an image. You have to enter the whole name of the image including the ending (e.g. ".png").

```
python3 image.py
```

The data created by the python script is added in the "image_data.h" file. Everytime you choose a new image, the data in this file will be overwritten.
After you generated the data for the image, you can compile the FFT code with the following

command:

```
riscv64-unknown-elf-gcc -static
-I./fftw-3.3.10/install/include
-L./fftw-3.3.10/install/lib
-o fftw.elf fftw.c -lfftw3 -lm
```

This will generate the executeable file for the simulation. Note that the conda environment needs to be activated and execute "source env.sh" in the chipyard directory. In the next step you can simulate this with spike:

```
spike pk fftw.elf
```

The output of the execution is written to the file "fft_output.txt", which contains a huge amount of lines.
The code in "gen_freq_pic.c" serves the visualiztion of the simulation output. Therefore first compile this code with (only neccesary once):

```
gen_freq_pic.c -o gen_freq_pic -lfftw3 -lm
```

And then execute with:

```
./gen_freq_pic
```

Edge Detection

To perform edge detection in simulation you need the following files in the same directory: image.py, edge.c, image_data.h, stb_image.h, stb_image_write.h and an image (like test.png).
The files stb_image.h and stb_image_write.h are added via:

```
wget https://raw.githubusercontent.com/nothings/stb/master/stb_image.h
wget https://raw.githubusercontent.com/nothings/stb/master/stb_image_write.h
```

The first step is to use the Python script to transform a picture into an array of values with the following command. You will be asked for a name of an image. You have to enter the whole name of the image including the ending, in this case the endig needs always to be ".jpg" for the simulation.

```
python3 image.py
```

The data created by the python script is added in the "image_data.h" file. Everytime you choose a new image, the data in this file will be overwritten.
After you generated the data for the image, you can compile the FFT code with the following command:

```
riscv64-unknown-elf-gcc -static  -o edge.elf edge.c  -lm
```

This will generate the executeable file for the simulation. Note that the conda environment needs to be activated and execute "source env.sh" in the chipyard directory. In the next step you can simulate this with spike:

```
spike pk edge.elf
```

The simulation will automatically generate a new image which shows the the result of the edge detection. The image for the result is saved in "edges.png".

Optical Flow

- measuring the movement of object in a scene, approximation of movement between to frames
- assumptions are neccesary:
1. brightness of an image point remains constant over time
2. displacemant and time step are very small
- maybe use of OpenCV
- Lucas Kanade Method: Assumption: for each pixel assume Motionfield (hence Optical Flow) is constant within a small neighborhood

Algorithms in Simulation with Verilator

Edge Detection

Configuration

To run edge detection in verilator it is to modify the code and the configuration of the rocket chip.
I created a new configuration for a rocket chip within the chipyard files I have on my local machine. Under the path "chipyard/generators/chipyard/src/main/scala/config" you can found several files which include differnt configurations for rocket chip. I added an own configuration in the file "MemorySystemConfigs.scala". The configuration I added looks like this:

```
class OwnMemoryRocketConfig extends Config( //ADDED BY ME
  new freechips.rocketchip.subsystem.WithoutTLMonitors ++
  new freechips.rocketchip.subsystem.WithExtMemSize((1<<30) * 1L) ++ // 1GB DRAM
  new freechips.rocketchip.rocket.WithNHugeCores(4) ++
  new freechips.rocketchip.subsystem.WithNMemoryChannels(4) ++
  new chipyard.config.AbstractConfig)
```

The configuration adds an external memory and several channels for this. Moreover this confi has more cores compared to the basic one of rocket chip with just one core. The first line helps to make the simulation faster (according to the configuration "FastRTLSimRocketConfig" in the file "RocketConfigs.scala").

Main Code

The modified code can be found in the directory "verilator_edge_dec" and is called "egdeBare.c". Compared to the code which was simulated with Spike, this code does not use system calls like malloc, calloc or free. These calls worked because Spike handles such things like this. But verilator simulates bare-metal conditions, that's why the modified code uses arrays. The calculation of edge detection is the same.
In the following you can see the result of executing the main code on an image with 159305786 simulation cycles (according to the ".out" file of the simulation):


Workflow

All of the code (the main code as well as the following code metioned) is located within the directory "chipyard/tests". Within this directory you can find the file "CMakeLists.txt". You need this file to add the follwing lines under the section "#Build":

```
add_executable(edgeBare edgeBare.c)
target_link_libraries(edgeBare m)
```

Before the simulation it is neccesary to prepare the data of an image for the main code. Therefore you can use the code "image.py" by running:

```
python3 image.py
```

You have to enter an image file and the width and heiht of this file, right now it is only possible to use ".jpg" files. The code will write the image data to "image_data.h" which is a two-dimensional array. After this it is possible to compile the testfiles for the execution in the same directory with the following commands.

```
cmake .

make
```

With the next command you can start the simulation with verilator. Therefore you have to change the directory to "chipyard/sims/verilator" and there you enter:

```
make run-binary CONFIG=OwnMemoryRocketConfig BINARY=../../tests/edgeBare.riscv LOADMEM=1
TIMEOUT_CYCLES=100000000000
```

Right now it is only possible to print the output in the terminal. To use the output after execution you have to manually add the following to the last command:

```
> ../../tests/output.txt
```

This will move the whole output to the "output.txt". With following command you can generate an image with the data written in "output.txt":

```
gcc generate_image.c -o generate_image

./generate_image
```

Further Code and Annotations

With the code in "generating.py" it is possible to generate simulated image data that is saved in "sim_data.h" automatically. The edge detection is currently working with this simulated data because it was not possible to run a simulation on real image data. Moreover it is not possible to save the output data in a certain file, it is just possible to print it in a console. With further work this will improve!
To generate simulated image data execute:

```
python3 generating.py
```

If you want to visualize the generated data you can execute the following commands and you will get an image based on the data:

```
gcc getData.c -o getData

./getData
```